

LexiPic Kiosk PWA

Technical Documentation

Version: 1.0.0

Type: Progressive Web App (static HTML/CSS/JavaScript, no build step, no server)

Requires: A modern browser; USB mode requires the File System Access API (Chromium-based browsers, v86+)

Publisher: BhoomiTech Heritage and Development Foundation

Part of: LexiPic Suite — see "LexiPic Suite Overview" for cross-module context

Table of Contents

Table of Contents.....	2
1. Introduction & Scope.....	3
1.1 Key capabilities	3
2. System Requirements.....	4
3. Architecture Overview	5
3.1 File structure	5
3.2 Screens	5
4. Source Modes	6
4.1 Remote URL mode.....	6
4.2 USB Drive mode.....	6
4.3 Switching modes	7
5. Service Worker & Offline Strategy	8
5.1 Install & activate	8
5.2 Fetch strategy.....	8
6. Security Model	9
6.1 Domain allowlist	9
6.2 Continuous re-validation	9
6.3 PIN-gated source changes.....	10
7. IndexedDB Schema	11
8. Data Formats (USB Mode)	12
8.1 settings.json.....	12
8.2 Per-set JSON files.....	12
9. Kiosk UI Behaviour (USB Mode)	14
10. Installation & Deployment	15
10.1 As a hosted web app (recommended for most deployments)	15
10.2 Opened directly from local files.....	15
10.3 Updating a deployed PWA.....	15
11. Operational Notes	16
12. Version Notes.....	17

1. Introduction & Scope

The LexiPic Kiosk PWA is the module of the LexiPic Suite that actually runs on kiosk hardware. It is a dependency-free, installable Progressive Web App — the application's own source comments describe it plainly: "No server. No PHP. Just open index.html." It has no build step, no package manager dependency, and no server-side component of its own.

This document covers its architecture: the two source modes a kiosk can be configured with, the offline-caching strategy, the IndexedDB schema, the security model that governs changing a kiosk's data source, and the file formats it consumes. For how this module fits alongside LexiPic (core) and LexiPic Kiosk (plugin), see the companion "LexiPic Suite Overview" document.

1.1 Key capabilities

- Two source modes, chosen once and remembered across reloads: Remote URL (displays a live WordPress-hosted kiosk page in a fullscreen iframe) and USB Drive (reads word-set data from a local folder with zero network dependency).
- A service worker that pre-caches the entire application shell on first load, so the app itself always starts even with no connectivity.
- In USB mode, a self-contained tile grid and card slider UI, rendered entirely client-side, with no network access required at any point — including on every subsequent boot.
- A domain allowlist check (fetched from the companion LexiPic Kiosk plugin) gating which URLs may be displayed in Remote URL mode, plus PIN-gated re-configuration that reuses the same plugin's existing kiosk PIN — no separate credential to manage.
- Continuous re-validation of an already-configured Remote URL against the live allowlist, so revoking a domain from wp-admin disconnects any kiosk currently displaying it, automatically.
- Installable as a true PWA (manifest.json + service worker) for a fullscreen, app-like presentation with no browser chrome.

2. System Requirements

Requirement	Value
Server	None. The PWA is a static file bundle — index.html, app.js, kiosk.css, sw.js, manifest.json, icons — that can be hosted anywhere or opened directly.
Browser — Remote URL mode	Any modern browser capable of displaying an <iframe> and registering a service worker.
Browser — USB Drive mode	A browser implementing the File System Access API (window.showDirectoryPicker) — currently Chromium-based browsers (Chrome, Edge) version 86 and later. Unsupported browsers see an explanatory message rather than a silent failure.
Network	Required continuously in Remote URL mode (it displays a live remote page). Required only for the very first setup and for periodic allowlist re-checks in USB mode — once configured, USB mode's core kiosk display works fully offline indefinitely.
Companion plugin	LexiPic Kiosk (WordPress plugin) — supplies the allowlist and PIN-verification endpoints this app calls. See the companion "LexiPic Kiosk (Plugin) — Technical Documentation."

3. Architecture Overview

3.1 File structure

lexipic-pwa/	
├ index.html	Setup, iframe, and kiosk-app screens
├ app.js	All application logic (1 file, no build step)
├ kiosk.css	Styling for every screen
├ manifest.json	PWA manifest (name, icons, display mode)
├ sw.js	Service worker – app-shell pre-caching
├ icons/	
│ └ icon-192.png	
│ └ icon-512.png	
└ lexipic-kiosk-data/	Example/reference USB-mode data folder
│ └ settings.json	Which sets to show + kiosk defaults
│ └ bm-dom-animal.json	One LexiPic-format set per file
│ └ bm-fish.json	
│ └ bm-fruits.json	
│ └ bm-vegetable.json	

Note: The bundled lexipic-kiosk-data/ folder ships as a worked example of the USB-mode data format; a real deployment supplies its own folder of the same shape, typically exported from LexiPic or LexiPic Kiosk.

3.2 Screens

Screen (DOM id)	Shown when
#screen-setup	On first launch, and whenever "Change source" is used — lets the operator choose Remote URL or USB Drive mode.
#screen-iframe	Remote URL mode is active — a fullscreen <iframe> displaying the configured page, plus a hidden error/retry overlay shown only on load failure.
#kiosk-root (#screen-sets / #screen-archive)	USB Drive mode is active — the self-rendered tile grid (#screen-sets) and card slider (#screen-archive), structurally similar to the LexiPic Kiosk plugin's own embedded kiosk markup.

A small "source" button is shown once a kiosk is active, the sole on-screen path back to the setup screen for reconfiguration.

4. Source Modes

4.1 Remote URL mode

Displays a remote web page — typically a LexiPic Kiosk plugin fullscreen URL such as `https://example.com/?lexipic_kiosk=1` — directly in a fullscreen `<iframe>`. The PWA does not parse, fetch, or understand anything about that page's content; it behaves like a locked-down kiosk browser pointed at a URL.

Behaviour	Detail
Persistence	The URL is saved in <code>localStorage</code> (key <code>lexipic_source</code>) and restored automatically on every reload — no re-entry needed after the first setup.
Failure handling	If the page fails to load (offline, DNS error, etc.), a small retry overlay (<code>#iframe-error</code>) appears; the app keeps attempting to reload it automatically every 8 seconds (<code>IFRAME_RETRY_MS</code>) until it succeeds, rather than giving up.
Security gating	The destination hostname must appear on the source WordPress site's domain allowlist (§6); changing an already-configured URL additionally requires the kiosk PIN (§6.3).
Network dependency	Required continuously — this mode has no offline capability of its own beyond the cached app shell (§5) and whatever the displayed remote page does for itself, e.g. the Kiosk plugin's own IndexedDB caching.

4.2 USB Drive mode

The operator selects a local `lexipic-kiosk-data` directory via the File System Access API (`showDirectoryPicker`). The directory must contain a `settings.json` manifest; the app then renders its own tile grid and card slider UI entirely client-side from that manifest and the per-set JSON files it lists — no network connection needed at any point once configured.

Behaviour	Detail
Handle persistence	The directory <code>HANDLE</code> itself (not just its name or path) is saved in IndexedDB, since <code>FileSystemDirectoryHandle</code> objects are structured-cloneable but not JSON-serializable. On reload, the app re-opens the saved handle and silently re-checks permission.
Re-grant flow	If the browser already granted folder access earlier, reopening needs no user interaction at all. If permission was revoked, or the handle has become unreadable, the setup screen asks for a single tap to re-grant access — never the full folder picker again, unless the saved handle is gone entirely (e.g. a different device).
Network dependency	None, once configured — this is the suite's fully offline-capable deployment path.
Security gating	No allowlist check applies (there is no remote URL to vet); changing an already-configured source still requires the kiosk PIN under the same "changing an existing source" rule as Remote URL mode (§6.3).

4.3 Switching modes

Both modes are reachable from the same setup screen at any time via the "☐ source" button, and a device can be reconfigured from one mode to the other freely — the PIN gate (§6.3) applies identically regardless of which mode is being changed to or from, since it protects "changing an already-configured source" as a single concept, not each mode individually.

5. Service Worker & Offline Strategy

sw.js implements a classic app-shell caching pattern: on install, every local asset plus the two Google Fonts stylesheets the app uses are added to a single named cache (lexipic-kiosk-v1).

```
const APP_SHELL = [
  './', './index.html', './kiosk.css', './app.js',
  './manifest.json', './icons/icon-192.png', './icons/icon-512.png',
  'https://fonts.googleapis.com/...' (Fraunces + Material Icons)
];
```

5.1 Install & activate

- On install, local app-shell files are cached first; the two external font stylesheets are cached in a second pass that is allowed to fail silently (`.catch(() => {})`), since fonts may be unreachable offline on first install but that must never block the app shell itself from being ready. `self.skipWaiting()` activates the new service worker immediately rather than waiting for all tabs to close.
- On activate, every cache not matching the current `CACHE_NAME` is deleted, and `self.clients.claim()` takes control of any already-open page immediately — together these ensure a deployed update to the PWA's own files fully replaces the previous cache on next load rather than mixing old and new assets.

5.2 Fetch strategy

The fetch handler is deliberately narrow in scope:

1. Any request whose origin is neither this app's own origin nor a `googleapis.com` font host passes straight through to the network, completely untouched by the service worker.
2. This specifically means the Remote-URL-mode iframe's page (a different origin, by design) and every sub-resource it loads are never intercepted or rewritten — the service worker must never interfere with a kiosk page being displayed in the iframe.
3. For app-shell requests, a cache-first strategy applies: a cached match is served immediately if present; otherwise the network is used and a successful (200) response is both returned and written into the cache for next time.

Scope of offline support: The service worker only ever caches the PWA's own shell. USB-mode word-set data (images/audio embedded as base64 in each set's JSON file) is handled entirely separately, by IndexedDB and the File System Access API — see §4.2 and §7.

6. Security Model

The PWA's security model addresses one specific question: can a kiosk be silently pointed at — or kept displaying — a page nobody approved? It is built entirely around reusing infrastructure the companion LexiPic Kiosk plugin already provides, rather than inventing a second, parallel credential system.

6.1 Domain allowlist

Any URL a person tries to save in Remote URL mode is checked against a domain allowlist fetched from the LexiPic Kiosk plugin's public GET /allowlist REST endpoint (configured in app.js as ALLOWLIST_ENDPOINT). Only an exact hostname match is accepted — no wildcard or subdomain inference.

Step	Behaviour
Fetch	getAllowlistData() requests the endpoint with cache: 'no-store'. On success, the response is cached locally (IndexedDB) and returned.
Fallback	On any fetch failure, the last successfully-cached copy is used instead, so a brief network blip doesn't block a legitimate source change.
Fail closed	If the endpoint cannot be reached AND nothing has ever been cached, the check returns "not allowed" with an explanatory reason — the action is blocked, never allowed through unchecked.
Match	checkUrlAllowed() compares the candidate URL's lowercased hostname against the cached domain list; anything not an exact match is rejected with a message naming the offending hostname.

6.2 Continuous re-validation

A saved Remote URL is not trusted indefinitely once accepted at save time. This is about integrity, not secrecy — the allowlist itself is not meant to be secret, but a saved source must never keep displaying once it is no longer approved, even if someone bypassed the save-time UI check directly (e.g. via browser developer tools).

- On every boot, bootUrlModeWithRevalidation() re-checks the saved URL's hostname against the live allowlist before displaying it.
- While running, startAllowlistWatchdog() re-checks periodically (ALLOWLIST_RECHECK_MS) in the background.
- Unlike the save-time check, periodic re-validation does NOT fail closed on an unreachable endpoint — a transient network blip must not turn into a full kiosk outage for a legitimately-saved URL. It only disconnects when the allowlist was actually reachable and explicitly does not include the current domain.
- The recheck is skipped if the iframe was loaded/reloaded too recently — the app's best available proxy for "someone is likely actively engaged with it," since a cross-origin iframe's internal clicks aren't observable to the parent page.
- When a periodic recheck does find a revoked URL, beginDisconnectCountdown() shows a short on-screen warning before disconnecting (disconnectRevokedUrl()), rather than vanishing instantly.

6.3 PIN-gated source changes

Changing an already-configured source — URL or USB — requires the kiosk PIN, verified by calling the LexiPic Kiosk plugin's existing POST /pin/verify endpoint (the exact same one its own on-screen admin panel uses). There is no separate, PWA-specific PIN.

Rule	Detail
First-ever setup	Not PIN-gated — <code>hasExistingSource()</code> is false (nothing saved in <code>localStorage</code> yet), so there is nothing to protect.
Changing a configured source	<code>promptForPin()</code> shows an inline overlay; the entered PIN is sent to <code>PIN_VERIFY_ENDPOINT</code> . A 429 response surfaces the rate-limit message; any other non-OK response is shown as "Incorrect PIN."
No offline PIN cache	Unlike the allowlist, the PIN check has no offline fallback at all — caching a PIN, even hashed, on the kiosk device would defeat the entire point of reusing a server-verified endpoint. A PIN change always requires network access.

7. IndexedDB Schema

Database LexipicKioskDB, version 3 (bumped from version 2 to add the allowlist-cache store). Three object stores:

Store	Key	Purpose
dir-handles	Fixed key lexipic-kiosk-data	The single USB-mode FileSystemDirectoryHandle, stored as a structured-cloned object (not JSON) so it can be reopened across reloads without a fresh folder picker.
allowlist-cache	Fixed key lexipic-kiosk-allowlist	The last successfully-fetched allowlist response (domains + metadata), used as the fail-open-on-network-blip / fail-closed-on-nothing-cached fallback described in §6.1.

Distinct from the Kiosk plugin's own cache: This is a separate IndexedDB database from the one the LexiPic Kiosk plugin's embedded app.js maintains (also named LexipicKioskDB, but populated independently in each context's own browser profile) — the PWA and a browser viewing the plugin's shortcode/fullscreen page do not share storage even on the same device, since they are different origins/contexts.

In USB mode specifically, the per-set JSON files themselves (including embedded base64 images/audio) are read directly from the granted directory handle on each access via the File System Access API — they are not separately duplicated into IndexedDB, since the directory handle already provides durable, permission-gated local access to the same files across reloads.

8. Data Formats (USB Mode)

USB mode reads two kinds of file from the selected directory: one settings.json manifest, and one JSON file per word set, in the same format LexiPic and LexiPic Kiosk both export.

8.1 settings.json

```
{
  "layers": [
    { "slug": "domestic-animals", "name": "Domestic Animals",
      "enabled": true, "group": "Nature" },
    { "slug": "numbers", "name": "Numbers 1-10",
      "enabled": true, "group": "Learning" },
    { "slug": "colours", "name": "Colours",
      "enabled": false, "group": "Learning" }
  ],
  "kiosk": {
    "idle_time_seconds": 90,
    "autoplay_audio": false
  }
}
```

Field	Meaning
layers[].slug	Maps to a {slug}.json file expected in the same directory.
layers[].name	Overrides the set's own internal name for display purposes.
layers[].enabled	false hides the set from the tile grid entirely without deleting the file.
layers[].group	A section-header label the tile grid groups sets under.
kiosk.idle_time_seconds / autoplay_audio	This device's starting defaults — equivalent in purpose to the LexiPic Kiosk plugin's site-wide Settings screen, but local to this USB drive / device.

8.2 Per-set JSON files

Each file listed in layers[].slug uses the same canonical export shape produced by LexiPic and consumed by LexiPic Kiosk (see the Suite Overview document, §4.1):

```
{
  "lexipic_version": "1.2.2",
  "set": { "name": "...", "slug": "...", "language": "bpm" },
  "entries": [
    {
      "id": "7",
      "word_script": "<word in native script>",
      "word_roman": "guru",
      "description": "...",
      "image": "data:image/jpeg;base64,...",
      "audio": "data:audio/webm;base64,..."
    }
  ]
}
```

Images and audio may be either embedded base64 data URIs (as above), or plain filenames relative to the data folder (e.g. "image": "cat.jpg") — `resolveMedia()` in `app.js` resolves either form, reading a referenced sibling file through the same directory handle when a value isn't already a data URI.

9. Kiosk UI Behaviour (USB Mode)

Once data loads, USB mode's UI is structurally similar to the LexiPic Kiosk plugin's own embedded kiosk — by design, since both descend from the same original standalone prototype's markup and behaviour.

Behaviour	Detail
Tile grid	<code>renderSetsGrid()</code> renders one tile per enabled set, grouped under its group label from <code>settings.json</code> .
Card slider	Opening a set calls <code>openSetArchive(slug)</code> , resolves every entry's media via <code>resolveMedia()</code> , and renders a horizontally-swipeable card deck (<code>renderCards()</code>) with Play/description per card, mirroring the plugin's own slider interaction (prev/next buttons, scroll-position-aware nav-button visibility).
Idle timer	<code>startKioskHeartbeat()</code> returns the device to the sets grid after <code>KIOSK_IDLE_MS</code> of no recorded activity (<code>handleUserActivity()</code> , bound to touch/click/scroll), matching the configured <code>idle_time_seconds</code> .
Autoplay	When <code>kioskAutoplay</code> is enabled, a card's audio plays automatically as it becomes the active card in the slider.
Load-error banner	If one or more sets listed in <code>settings.json</code> fail to load (e.g. a missing or malformed <code>{slug}.json</code>), <code>renderLoadErrorBanner()</code> surfaces which ones, rather than silently omitting them with no explanation.

10. Installation & Deployment

10.1 As a hosted web app (recommended for most deployments)

4. Host the `lexipic-pwa/` folder's contents on any static web server or CDN (no server-side processing is required).
5. Navigate to it on the kiosk device; the browser will offer to install it as an app (`manifest.json` declares `display: "fullscreen"`).
6. On first launch, choose Remote URL or USB Drive mode from the setup screen.
7. For Remote URL mode, ensure the target domain has already been added to the companion LexiPic Kiosk plugin's Approved Kiosk URLs page (see that document, §12) — otherwise the save will be rejected.

10.2 Opened directly from local files

Because the app has no server dependency, `index.html` can also be opened directly from a local copy of the folder (e.g. via a `file://` URL or a minimal local static server) on the kiosk device itself, which is the typical pattern for a fully offline USB-mode kiosk that should never depend on any external hosting.

10.3 Updating a deployed PWA

Because the service worker pre-caches the app shell aggressively (§5), a kiosk that has already loaded the app once will continue running its currently-cached version until the service worker's install/activate cycle replaces the cache — typically the next time the app is opened while the underlying hosted files have changed. There is no separate auto-update mechanism beyond the browser's normal service-worker update check.

11. Operational Notes

- USB mode requires a Chromium-based browser (Chrome or Edge, version 86+) for the File System Access API; on an unsupported browser the relevant setup button explains this rather than failing silently.
- A directory handle's permission can be revoked by the operating system or browser independently of the app (e.g. after an OS update, or if the underlying drive is removed and reattached) — the app detects this and asks for a single re-grant tap rather than the full folder picker, unless the handle itself becomes entirely invalid.
- Remote URL mode has no offline capability of its own; if the kiosk loses connectivity, the iframe shows the retry overlay until connectivity returns — the app does not fall back to USB-style local data in that mode.
- Because the PIN check has no offline cache (§6.3), a kiosk operator cannot change an existing source while genuinely offline, by design.
- The bundled `lexipic-kiosk-data/` example folder is reference material, not live production data — replace it with an export from LexiPic or LexiPic Kiosk for an actual deployment.

12. Version Notes

Version	Notes
1.0.0	Initial release. Two source modes (Remote URL, USB Drive); service-worker app-shell caching; domain allowlist + kiosk-PIN reuse for source-change security; IndexedDB-backed directory-handle and allowlist caching.